TEMA

Tema 9. Utilización avanzada de clases

Informática

Programación I

Autoría: Cristian Catalán

Linkia FP

Tema 9: Utilización avanzada de clases

¿Qué aprenderás?

- Identificar el polimorfismo.
- Crear y utilizar interfaces y clases abstractas para utilizar el polimorfismo.

¿Sabías que…?

- Ad hoc polymorphism es el concepto acuñado en 1968 (y utilizada en el lenguaje Algol 68) para describir la capacidad de que una misma función pudiera recibir parámetros de distintos tipos.
- En Java utilizamos el concepto de polimorfismo llamado *subtype polymorphism* or *inclusion polymorphism*.
- Según Tiboe, Java es el lenguaje de programación más popular.

VERSIÓN IMPRIMIBLE PARA ALUMNO LINKIAFP

1. Polimorfismo

El Polimorfismo es una característica de la POO que nos va a permitir ejecutar un mismo código con variables de distinto tipo (pero con algunas restricciones). El polimorfismo es una característica que tenemos junto con la Herencia, pero que también la veremos más a delante con las clases Abstractas y las Interfaces.

Para que pueda ejecutar un mismo método sobre objetos de distinto tipo necesito que todos estos objetos hereden de una misma superclase que tenga definida ese método. Además, como una variable solo puede ser de un tipo, la variable que ejecute el método va a ser del tipo de la superclase.

Podemos pensar que lo que se está produciendo es un *cast* de la subclase para obtener una variable de la superclase, de forma que todos los atributos y métodos que sólo existan en la subclase y no estén presentes en la superclase van a ser eliminados.

Por ejemplo, si tenemos una variable de la clase AlimentoSolido la vamos a poder guardar en una variable de la superclase Alimento conservando solo aquellos atributos y métodos comunes.

AlimentoSolido bistec = new AlimentoSolido("bistec",100,400,20);

Alimento soloAlimento = (Alimento)bistec;

Aunque no hace falta especificar el *cast* añadiendo (Alimento) delante de la variable, nos sirve para entender la acción.

En el siguiente ejemplo hemos definido una función *imprimeAlimento(Alimento alim)* que recibe un como parámetro un objeto del tipo *Alimento* y utiliza su función *toString()* para mostrar la información del alimento por consola. En el main declaramos un objeto de tipo *Alimento,* otro de tipo *AlimentoSolido* y otro de tipo *AlimentoLiquido* y llamamos a *imprimeAlimento()* con los tres objetos obteniendo como resultado los mensajes:

nombre:naranjada,cantidad:100

nombre:bistec,cantidad:100gramos,dureza:20

nombre:limonada,cantidad:100litros,viscosidad:9

VERSIÓN IMPRIMIBLE ALUMNO/A TUTORIAL

```
public class EjemploHerencia {
   public static void main(String[] args) {
      Alimento naranjada = new Alimento("naranjada",100,35);
      AlimentoSolido bistec = new AlimentoSolido("bistec",100,
                                                        400,20);
      AlimentoLiquido limonada = new AlimentoLiquido(
                                          "limonada",100,25,9);
      imprimeAlimento(naranjada);
      imprimeAlimento(bistec);
      imprimeAlimento(limonada);
}

   static void imprimeAlimento(Alimento alim){
      System.out.println(alim.toString());
   }
}
```

```
public class Alimento {
   String nombre;
   int cantidad;
   String unidadMedida=" ";
   int kcal=0;
   Alimento(){ }
   Alimento(String nombre, int cantidad, int kcal) {
     this.nombre=nombre;
     this.cantidad=cantidad;
     this.kcal= kcal;
   }
   public String toString(){
      return "nombre:"+nombre+
            ",cantidad:"+cantidad+unidadMedida;
   }
}
```

```
public class AlimentoSolido extends Alimento {
   int dureza;
   String unidadMedida="gramos";

   public AlimentoSolido(String bistec, int cantidad,
                              int kcal, int dureza) {
      super(bistec,cantidad,kcal);
      this.dureza = dureza;
   }

   public String toString(){
      return "nombre:"+nombre+
            ",cantidad:"+cantidad+unidadMedida+
            ",dureza:"+dureza;
   }
}
```

```
public class AlimentoLiquido extends Alimento {
   int viscosidad;
   String unidadMedida="litros";

   public AlimentoLiquido(String bistec, int cantidad,
                              int kcal, int viscosidad) {
      super(bistec,cantidad,kcal);
      this.viscosidad = viscosidad;
   }

   public String toString(){
      return "nombre:"+nombre+
            ",cantidad:"+cantidad+unidadMedida+
            ",viscosidad:"+viscosidad;
   }
}
```

Ejemplo de polimorfismo con herencia

Como podemos observar en el ejemplo anterior las implementaciones de las funciones comunas con la superclase se mantienen tal y como se han sobrescrito en la subclase. Por ello la función toString ejecuta un resultado distinto según el objeto recibido.

Observando el ejemplo anterior podríamos pensar que crear un objeto de la clase Alimento va a tener muy poco sentido, ya que un alimento va a ser o de tipo AlimentoLiquido o de tipo AlimentoSolido (podríamos añadir otro de gaseoso, pero no lo complicaremos más). Pero sí que es necesaria para que podamos derivar de ella otras subclases y poder utilizar el polimorfismo tal como lo hemos utilizado en el ejemplo.

Si nos encontramos en esta situación, con una clase de la no tiene sentido que se pueda inicializar ningún objeto, significa que ésa clase va a ser Abstracta.

2. Clases Abstractas

Es el vecino al que escoge al alcalde y es el alcalde el que quiere... en fin...

Una clase abstracta define una estructura de datos para ser usada sólo como superclase impidiendo así que pueda crear una instancia (un objeto) de una clase abstracta. Como nunca va a poder existir un objeto a partir de una clase abstracta, ésta va a poder contener métodos sin estar implementados que declararemos como "abstractos".

Un método abstracto va a ser un método que no está implementado, sólo declarado indicando el nombre, parámetros y tipo de retorno. Solo una clase abstracta va a poder contener métodos abstractos. Y aquellas clases que hereden de una clase abstracta van a tener que implementar sus métodos abstractos.

Una clase abstracta puede tener también métodos abstractos y no abstractos.

Aunque no podemos inicializar una variable a partir de una clase abstracta, sí que vamos a poder declarar una variable de una clase abstracta.

Una variable declarada del tipo de una clase abstracta sólo va a poder ser inicializada a partir de una subclase gracias al polimorfismo. Por ejemplo, si la clase *Alimento* es abstracta y la clase *AlimentoSolido* es una subclase, vamos a poder crear un objeto del tipo *Alimento* inicializándola con *AlimentoSolido* de la siguiente manera:

Alimento alimGenerico = new AlimentoSolido("bistec", 100, 400, 20);

Pero recordar que *alimGenerico* solo contendrá la estructura de datos definida en Alimento, aunque su implementación y valores serán los definidos en *AlimentoSolido*.

Java utiliza el modificador **abstract** para definir una clase o método como abstracto. La sintaxis es:

abstract *class NombreClase {.....}*

abstract *RetornoFuncion nombreFuncion();*

VERSIÓN MEDIANTE EL CURSO ANKIAFP

En el siguiente ejemplo hemos definido como abstracta a la clase *Alimento* y a su método *imprimir*. Hemos creado un array de *Alimento* rellenado con un objeto del tipo *AlimentoSolido* y otro *AlimentoLiquido*. Como la función *imprimir*() está definida en la clase abstracta, desde la función *imprimeAlimentos*() la hemos podido llamar para cada elemento del array ejecutándose la implementación distinta que contiene cada subclase. Por consola muestra el siguiente mensaje:

nombre:bistec,cantidad:100gramos,dureza:20

nombre:limonada,cantidad:100litros,viscosidad:9

```
public class EjemploHerencia {
   public static void main(String[] args) {
      AlimentoSolido bistec = new AlimentoSolido("bistec",100,
                                                 400,20);
      AlimentoLiquido limonada = new AlimentoLiquido(
                                 "limonada",100,25,9);
      Alimento[] alimentos = new Alimento[2];
      alimentos[0] = bistec;
      alimentos[1] = limonada;
     imprimeAlimentos(alimentos);
}
   static void imprimeAlimentos(Alimento[] alim) {
      for(int k=0;k<alim.length;k++){
         System.out.println(alim[k].imprimir ());
      }
   }}
```

```
public abstract class Alimento {
   String nombre;
   int cantidad;
   String unidadMedida=" ";
   int kcal=0;
   Alimento(){ }
   Alimento(String nombre, int cantidad, int kcal) {
     this.nombre=nombre;
     this.cantidad=cantidad;
     this.kcal= kcal;
   }
  public abstract String imprimir() ;
}
```

```
public class AlimentoSolido extends Alimento {
   int dureza;
   String unidadMedida="gramos";

   public AlimentoSolido(String bistec, int cantidad,
                         int kcal, int dureza) {
      super(bistec,cantidad,kcal);
      this.dureza = dureza;
   }

   public String imprimir (){
      return "nombre:"+nombre+
           ",cantidad:"+cantidad+unidadMedida+
            ",dureza:"+dureza;
   }
}
```

```
public class AlimentoLiquido extends Alimento {
    int viscosidad;
   String unidadMedida="litros";

   public AlimentoLiquido(String bistec, int cantidad,
                          int kcal, int viscosidad) {
      super(bistec,cantidad,kcal);
      this.viscosidad = viscosidad;
   }

   public String imprimir (){
      return "nombre:"+nombre+
           ",cantidad:"+cantidad+unidadMedida+
            ",viscosidad:"+viscosidad;
   }
}
```

Alimento es una clase abstracta con un método toString() también abstracto

Si recuerdas ejemplos anteriores, verás que la función que ahora hemos llamado *imprimir* antes la habíamos llamado *toString*. El cambio se debe a que como recordaras *toString* es una función de *Object*, y para el ejemplo hemos preferido declarar un método exclusivo de *Alimento* como abstracto para obligar a que las clases derivadas lo deban implementar.

Ahora vamos a pensar si sería posible en vez de crear una función *imprimeAlimentos*, crear una función que pudiera llamar a la función imprimir de cualquier clase que la tuviera definida aunque no compartiesen superclase. Para conseguirlo vamos a tener que definir una interfaz con la función imprimir e indicar a la clase que quiera imprimir que utilice ésa interfaz. Vamos a verlo en el siguiente punto.

3. Interfaces

Las interfaces no permiten agrupar un conjunto de declaraciones de métodos y constantes.

En java podemos definir una interfaz con la palabra clave "interface". Por ejemplo, vamos a declarar la interfaz Imprimible que define el método *imprimir* y la constante AUTOR con el valor "Cristian".

```
public interface Imprimible {
   public static String AUTOR="Cristian";
   public String imprimir();
}
```

Los atributos de una interfaz siempre serán del tipo **final** y por lo tanto su valor no se va a poder modificar. Las funciones en una interfaz nunca se van a definir, solo se declaran.

Cualquier clase va a poder utilizar una o varias interfaces. Diremos entonces que una clase implementa ésas interfaces.

Cualquier clase que implemente una interfaz va a tener que implementar todas las funciones definidas en la interfaz.

Como ocurre con las clases abstractas, no vamos a poder inicializar un objeto a partir de una interfaz, solo lo podremos declarar. La inicialización de ése objeto lo deberá hacer alguna clase que implemente ésa interfaz. Por ejemplo, si *AlimentoLiquido* implementa *Imprimible*, puedo declarar una variable del tipo *Imprimible* e inicializar el objeto a partir de *AlimentoLiquido*:

Imprimible alimento1 = new AlimentoLiquido();

El nombre de una interfaz suele ser un adjetivo ya que nos servirá para indicar que una clase va a poder realizar los métodos definidos en la interfaz.

Si trabajamos con objetos de un cierto tipo de interfaz vamos a poder inicializar esos objetos con cualquier clase que utilice ésa interfaz. Por ejemplo, en el siguiente código hemos creado la interfaz *Imprimible* y desarrollado en la clase ejecutable una función que va a poder llamar al método imprimir de cualquier objeto pasado por parámetro que sea del tipo Imprimible. También hemos declarado una nueva clase Cubierto que no comparte superclase con las clases AlimentoSolido y AlimentoLiquido, pero las tres implementan la interfaz Imprimible. Ejecutar el siguiente código mostrará por consola los valores:

nombre:bistec,cantidad:100gramos,dureza:20

nombre:limonada,cantidad:100litros,viscosidad:9

nombre:cuchara, funcion: comer liquidos

VERSIÓN IMPRIMIBLE ALUMNO LINKIAFP

```
public class EjemploHerencia {

   public static void main(String[] args) {
      AlimentoSolido bistec = new AlimentoSolido("bistec", 100
                                                        , 400, 20);
      AlimentoLiquido limonada = new AlimentoLiquido("limonada"
                                                        , 100, 25, 9);
      Cubierto cuchara = new Cubierto("cuchara"
                                                  ,"comer liquidos");

      imprimeCualquierImprimible(bistec);
      imprimeCualquierImprimible(limonada);
      imprimeCualquierImprimible(cuchara);
   }
   static void imprimeCualquierImprimible(Imprimible im){
      String texto= im.imprimir();
      System.out.println(  texto );
   }
}
```

```
public interface Imprimible {
   public static String AUTOR="Cristian";
   public String imprimir();
}
```

```
public class Cubierto implements Imprimible{
   String nombre;
   String funcion;
   public Cubierto(String nom, String fun){
      this.nombre=nom;
      this.funcion=fun;
   }
   public String imprimir() {
      return "nombre:"+nombre+
            ", funcion: "+funcion;
   }
}
```

```
public class AlimentoSolido
                extends Alimento implements Imprimible{
   int dureza;
   String unidadMedida="gramos";

   public AlimentoSolido(String bistec, int cantidad,
                                    int kcal, int dureza) {
      super(bistec,cantidad,kcal);
      this.dureza = dureza;
   }
   public String imprimir(){
      return "nombre:"+nombre+
            ",cantidad:"+cantidad+unidadMedida+
             ",dureza:"+dureza;
   }
}
```

```
public class AlimentoLiquido
            extends Alimento implements Imprimible {
   int viscosidad;
   String unidadMedida="litros";

   public AlimentoLiquido(String bistec, int cantidad,
                                    int kcal, int viscosidad) {
      super(bistec,cantidad,kcal);
      this.viscosidad = viscosidad;
   }
   public String imprimir(){
      return "nombre:"+nombre+
            ",cantidad:"+cantidad+unidadMedida+
             ",viscosidad:"+viscosidad;
   }
}
```

Vínculo: Herencia_Polimorfismo: Ejemplo de polimorfismo con interfaces

VERSIÓN IMPRIMIBLE ALUMNO LINK

Recursos y enlaces

- Documentación oficial sobre el polimorfismo:
  https://docs.oracle.com/javase/tutorial/java/IandI/polymorphism.html
- Documentación oficial sobre clases abstractas:
  https://docs.oracle.com/javase/tutorial/java/IandI/abstract.html
- Documentación oficial sobre interfaces:
  https://docs.oracle.com/javase/tutorial/java/IandI/createinterface.html

VERSIÓN IMPRIMIBLE ALUMNO LINKIAFP

Conceptos clave

- **polimorfismo**: capacidad de almacenar en un objeto una instancia de otro objeto de una clase hija.
- **abstracta**: una clase de la que no se puede crear una instancia.
- **interfaz**: define un conjunto de características que ha de tener una clase.

Test de autoevaluación

¿Las clases abstractas pueden contener métodos implementados?

a) Sí, siempre.

b) No, nunca.

c) Sí, siempre que no se indiquen como métodos abstractos.

d) No, excepto los constructores.

¿Las interfaces pueden contener métodos implementados?

a) Sí, siempre.

b) No, nunca.

c) Sí, siempre que no se indiquen como métodos abstractos.

d) No, excepto los constructores.

VERSIÓN IMPRIMIBLE ALUMNO LINKIAFP

Ponlo en práctica

Actividad 1

Vamos a seguir con la actividad del tema anterior que dice:

"Queremos estructurar la información de los habitantes de una ciudad extraterrestre formada por dos razas distintas: los Cabecines y los Cabezones. De cada individuo guardaremos su nombre y su color de piel (los individuos de ambas razas pueden tener nombres y colores distintos al resto). A demás queremos guardar para cada raza su nombre y el número de espacios que ocupan teniendo en cuenta que los Cabecines ocupan 1 espacio y los Cabezones ocupan 2.

Ambas razas van a tener una función de nombre *toString()* que retorne una string con los atributos del individuo.

Ambas razas van a tener una función de nombre *estresar()* que cuando se ejecute cambiaré el color de piel de los Cabezones a azul y el color de piel de los Cabecines a rojo.

Añade ahora a los Cabezones un tercer ojo que puede ser "marron", "negro" o "rojo".

Crea la superclase Extraterrestre de la que hereden Cabecin y Cabezon. La clase Extraterrestre tendrá el atributo *raza* con valor "extraterrestre", el atributo *espacio* con valor inicial de 1.

Crea ahora una clase principal de nombre "CabezaWordl" que al ejecutarse cree 2 Cabecines y 2 Cabezones estableciendo su nombre y color con un constructor.

En el caso de los Cabezones establece también el color de su tercer ojo mediante un setter que establezca el tercer ojo como "rojo" si no se introduce un valor correcto.

Ahora como *Extraterrestre* no tiene nada implementado lo transformaremos en una clase abstracta. Crearemos también una clase *Paquete* con el atributo "espacio" y "descripción". Define en *CabezaWorld* un array "habitantes" que contenga Cabecines y Cabezones y posteriormente imprime la información de los "habitantes".

Utilizando una interfaz de nombre "OcupaEspacio" crearemos en "CabezaWorld" un array "autobús" que pueda contener elementos de la clase Cabecin, Cabezon o Paquete. Para conseguirlo las 3 clases deberán implementar "OcupaEspacio". Muestra a continuación la descripción y el espacio que ocupan todos los integrantes del autobús.

VERSIÓN IMPRIMIBLE ALUMNO LICKAEP

SOLUCIONARIOS

Test de autoevaluación

¿Las clases abstractas pueden contener métodos implementados?

a) Sí, siempre.

b) No, nunca.

c) Sí, siempre que no se indiquen como métodos abstractos.

d) No, excepto los constructores.

¿Las interfaces pueden contener métodos implementados?

e) Sí, siempre.

f) No, nunca.

g) Sí, siempre que no se indiquen como métodos abstractos.

h) No, excepto los constructores.

VERSIÓN IMPRIMIBLE ALUMNO LINKIAFP

Ponlo en práctica

Actividad 1

Vamos a seguir con la actividad del tema anterior que dice:

"Queremos estructurar la información de los habitantes de una ciudad extraterrestre formada por dos razas distintas: los Cabecines y los Cabezones. De cada individuo guardaremos su nombre y su color de piel (los individuos de ambas razas pueden tener nombres y colores distintos al resto). A demás queremos guardar para cada raza su nombre y el número de espacios que ocupan teniendo en cuenta que los Cabecines ocupan 1 espacio y los Cabezones ocupan 2.

Ambas razas van a tener una función de nombre *toString()* que retorne una string con los atributos del individuo.

Ambas razas van a tener una función de nombre *estresar()* que cuando se ejecute cambiaré el color de piel de los Cabezones a azul y el color de piel de los Cabecines a rojo.

Añade ahora a los Cabezones un tercer ojo que puede ser "marron", "negro" o "rojo".

Crea la superclase Extraterrestre de la que hereden Cabecin y Cabezon. La clase Extraterrestre tendrá el atributo *raza* con valor "extraterrestre", el atributo *espacio* con valor inicial de 1.

Crea ahora una clase principal de nombre "CabezaWordl" que al ejecutarse cree 2 Cabecines y 2 Cabezones estableciendo su nombre y color con un constructor.

En el caso de los Cabezones establece también el color de su tercer ojo mediante un setter que establezca el tercer ojo como "rojo" si no se introduce un valor correcto.

Ahora como *Extraterrestre* no tiene nada implementado lo transformaremos en una clase abstracta. Crearemos también una clase *Paquete* con el atributo "espacio" y "descripción". Define en *CabezaWorld* un array "habitantes" que contenga Cabecines y Cabezones y posteriormente imprime la información de los "habitantes".

Utilizando una interfaz de nombre "OcupaEspacio" crearemos en "CabezaWorld" un array "autobús" que pueda contener elementos de la clase Cabecin, Cabezon o Paquete. Para conseguirlo las 3 clases deberán implementar "OcupaEspacio". Muestra a continuación la descripción y el espacio que ocupan todos los integrantes del autobús.

Los solucionarios están disponibles en la versión interactiva del aula.

VERSIÓN IMPRIMIBLE AULA ONLINEA